

この章では、レシーバーを別の種類の関数であるラムダ式と組み合わせて使用する方法、およびそれらがドメイン固有言語（DSL）の作成にどのように役立つかを学びます。

Lambda expressions with receiver

レシーバーを持つラムダ式

初心者向けツアーでは、[ラムダ式](#)の使い方を学びました。ラムダ式には、レシーバーを指定することもできます。この場合、ラムダ式は、その都度レシーバーを明示的に指定することなく、レシーバーの任意のメンバ関数やプロパティにアクセスすることができます。こうした余分な参照がなければ、コードは読みやすく、メンテナンスも容易になります。

レシーバーを持つラムダ式は、レシーバーを持つ関数リテラルとも呼ばれます。

{style="tip"}

レシーバーを持つラムダ式の構文は、関数型を定義する場合とは異なります。まず、拡張したいレシーバーを記述します。次に、`.`を入力し、関数型の定義の残りの部分を完成させてください。例えば：

```
MutableList<Int>().() -> Unit
```

この関数型には、以下のものがあります：

- レシーバーとして `MutableList<Int>`。
- 括弧 `()` 内に関数の引数を含めないでください。
- 戻り値なし：`Unit`。

キャンバス上に図形を描く次の例を考えてみましょう：

```
class Canvas {
    fun drawCircle() = println("🟡 Drawing a circle")
    fun drawSquare() = println("🟠 Drawing a square")
}

// レシーバー定義を含むラムダ式
fun render(block: Canvas.() -> Unit): Canvas {
    val canvas = Canvas()
    // レシーバー付きのラムダ式を使用する
    canvas.block()
    return canvas
}

// 訳者注
// render関数のblockはCanvasクラスの関数を受け取る

fun main() {
    render {
        drawCircle()
        // 🟡 Drawing a circle
        drawSquare()
    }
}
```

```
//  Drawing a square
    }
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-intermediate-tour-lambda-expression-with-receiver"}
```

この例では：

- `Canvas` クラスには、円や四角形を描くことをシミュレートする 2 つの関数があります。
- `render()` 関数は `block` パラメータを受け取り、`Canvas` クラスのインスタンスを返します。
- `block` パラメータは、レシーバーを持つラムダ式であり、そのレシーバーは `Canvas` クラスです。
- `render()` 関数は、`Canvas` クラスのインスタンスを作成し、その `canvas` インスタンスをレシーバーとして、`block()` ラムダ式を呼び出します。
- `main()` 関数は、`block` パラメータに渡されるラムダ式を引数として `render()` 関数を呼び出します。
- `render()` 関数に渡されたラムダ式内部で、プログラムは `Canvas` クラスのインスタンスに対して `drawCircle()` および `drawSquare()` 関数を呼び出します。
- `render()` 関数に渡されたラムダ式内部で、プログラムは `Canvas` クラスのインスタンスに対して `drawCircle()` および `drawSquare()` 関数を呼び出します。

レシーバーを持つラムダ式は、ドメイン固有言語（DSL）を作成したい場合に役立ちます。レシーバーを明示的に参照することなく、そのメンバー関数やプロパティにアクセスできるため、コードが簡潔になります。

これを説明するために、メニュー内の項目を設定する例を考えてみましょう。まずは、`MenuItem` クラスと、メニューに項目を追加する `item()` という関数、およびすべてのメニュー項目の一覧 `items` を含む `Menu` クラスから始めましょう：

```
class MenuItem(val name: String)

class Menu(val name: String) {
    val items = mutableListOf<MenuItem>()

    fun item(name: String) {
        items.add(MenuItem(name))
    }
}
```

まずは、メニューを構築する `menu()` 関数に、関数パラメータ（`init`）としてレシーバーが渡されるラムダ式を、出発点として使ってみましょう：

```
fun menu(name: String, init: Menu.() -> Unit): Menu {
    // Menuクラスのインスタンスを作成します
```

```
    val menu = Menu(name)
    // クラスインスタンスに対して、レシーバー init() を持つラムダ式を呼び出します
    menu.init()
    return menu
}
```

これで、DSL を使ってメニューを設定し、メニュー構造をコンソールに出力する `printMenu()` 関数を作成できるようになりました：

```
class MenuItem(val name: String)

class Menu(val name: String) {
    val items = mutableListOf<MenuItem>()

    fun item(name: String) {
        items.add(MenuItem(name))
    }
}

fun menu(name: String, init: Menu.() -> Unit): Menu {
    val menu = Menu(name)
    menu.init()
    return menu
}

//sampleStart
fun printMenu(menu: Menu) {
    println("Menu: ${menu.name}")
    menu.items.forEach { println("  Item: ${it.name}") }
}

// Use the DSL
fun main() {
    // Create the menu
    val mainMenu = menu("Main Menu") {
        // Add items to the menu
        item("Home")
        item("Settings")
        item("Exit")
    }

    // Print the menu
    printMenu(mainMenu)
    // Menu: Main Menu
    //   Item: Home
    //   Item: Settings
    //   Item: Exit
}
//sampleEnd
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-intermediate-tour-lambda-expression-with-receiver-dsl"}
```

ご覧のとおり、レシーバーを伴うラムダ式を使用することで、メニューを作成するために必要なコードが大幅に簡略化されます。ラムダ式は、セットアップや生成だけでなく、設定にも役立ちます。これらは、API、UIフレームワーク、設定ビルダー向けのDSLを構築する際に広く利用されており、コードを簡潔にすることで、基盤となるコードの構造やロジックに集中しやすくなります。

Kotlinのエコシステムには、このデザインパターンの例が数多く見られます。例えば、`buildList()` や `buildString()` (<https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.text/build-string.html>) 関数など、このデザインパターンの例が数多く見られます。

Kotlinでは、レシーバーを持つラムダ式を**型安全なビルダー**と組み合わせることで、実行時ではなくコンパイル時に型の問題を検出できるDSLを作成できます。詳細については、[型安全なビルダー](#)を参照してください。

```
{style="tip"}
```

演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="lambda-receivers-exercise-1"}

`fetchData()` という関数があり、この関数はレシーバー付きのラムダ式を受け取ります。コードの出力が `Data received - Processed` になるよう、ラムダ式を修正して `append()` 関数を使用するようにしてください。

```
|---|---|
```

```
fun fetchData(callback: StringBuilder.() -> Unit) {
    val builder = StringBuilder("Data received")
    builder.callback()
}

fun main() {
    fetchData {
        // Write your code here
        // Data received - Processed
    }
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-receivers-exercise-1"}
```

```
|---|---|
```

```
fun fetchData(callback: StringBuilder.() -> Unit) {
    val builder = StringBuilder("Data received")
    builder.callback()
}
```

```
fun main() {  
    fetchData {  
        append(" - Processed")  
        println(this.toString())  
        // Data received - Processed  
    }  
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-lambda-receivers-solution-1"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="lambda-receivers-exercise-2"}

`Button` クラスと、`ButtonEvent` および `Position` というデータクラスがあります。ダブルクリックイベントを発生させるために、`Button` クラスの `onEvent()` メンバ関数を呼び出すコードを書いてください。このコードを実行すると、「`Double click!`」と表示されるはずです。

```
class Button {  
    fun onEvent(action: ButtonEvent.() -> Unit) {  
        // ダブルクリックイベントをシミュレートする（右クリックではない）  
        val event = ButtonEvent(isRightClick = false, amount = 2, position =  
Position(100, 200))  
        event.action() // イベントのコールバックをトリガーする  
    }  
}  
  
data class ButtonEvent(  
    val isRightClick: Boolean,  
    val amount: Int,  
    val position: Position  
)  
  
data class Position(  
    val x: Int,  
    val y: Int  
)  
  
fun main() {  
    val button = Button()  
  
    button.onEvent {  
        // Write your code here  
        // Double click!  
    }  
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-receivers-exercise-2"}

|---|---|

```

class Button {
    fun onEvent(action: ButtonEvent.() -> Unit) {
        // ダブルクリックイベントをシミュレートする（右クリックではない）
        val event = ButtonEvent(isRightClick = false, amount = 2, position =
Position(100, 200))
        event.action() // Trigger the event callback
    }
}

data class ButtonEvent(
    val isRightClick: Boolean,
    val amount: Int,
    val position: Position
)

data class Position(
    val x: Int,
    val y: Int
)

fun main() {
    val button = Button()

    button.onEvent {
        if (!isRightClick && amount == 2) {
            println("Double click!")
            // Double click!
            println("on $position")
        }
    }

    // 訳者注)
    // Buttonクラスのインスタンスbuttonを通して、
    // ButtonEventクラスに
    // 新たな関数(機能)
    // {
    //     if (!isRightClick && amount == 2) {
    //         println("Double click!")
    //     }
    // } を
    // 追加する。
    // この機能は、ButtonクラスのonEvent()関数の引数actionに渡される。
    // 引数actionは、ButtonEventクラスの()->Unitの型を持った関数である。
    // event.action()で実行される。
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-lambda-receivers-solution-2"}

課題 3 {initial-collapse-state="collapsed" collapsible="true" id="lambda-receivers-exercise-3"}

各要素を1ずつ増やした整数のリストのコピーを作成する関数を作成してください。 `List<Int>` に `incremented` 関数を追加する、提供されている関数の骨格を使用してください。

```
fun List<Int>.incremented(): List<Int> {
    val originalList = this
    return buildList {
        // Write your code here
    }
}

fun main() {
    val originalList = listOf(1, 2, 3)
    val newList = originalList.incremented()
    println(newList)
    // [2, 3, 4]
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-lambda-receivers-exercise-3"}

|---|---|

```
fun List<Int>.incremented(): List<Int> {
    val originalList = this
    return buildList {
        for (n in originalList) add(n + 1)
    }
}

fun main() {
    val originalList = listOf(1, 2, 3)
    val newList = originalList.incremented()
    println(newList)
    // [2, 3, 4]
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-lambda-receivers-solution-3"}

- [Previous step](#)
- [Next step](#)